CineFlix

SQL Analytics Project

End-to-End Business Intelligence Case Study

Prepared for: CineFlix Data Analytics Team

Database: MySQL (Sakila-derived schema)

Version: 1.0 | June 2025

1. Executive Summary

CineFlix is a multi-store DVD rental business operating across several cities. This document presents a comprehensive SQL analytics case study covering the complete database schema, all entity relationships, and ten targeted business queries that extract actionable insights from the CineFlix data warehouse.

Business Objectives:

- Identify top-performing film genres by rental volume to guide inventory procurement.
- Segment customers by rental activity and spending to prioritize marketing spend.
- Optimize inventory allocation by matching supply to demand at the store level.
- Monitor staff performance and detect revenue leakage via unprocessed returns.
- Benchmark film pricing against the portfolio average to enable dynamic pricing.

Schema :-

15	3	10	68+
Tables	Domains	SQL Queries	Total Columns

2. Database Schema Diagram

The CineFlix schema is organized into three functional domains: Customer Data (pink), Business Operations (green), and Movie Inventory (blue). Foreign key relationships are represented by dashed lines with crow's foot notation indicating cardinality.

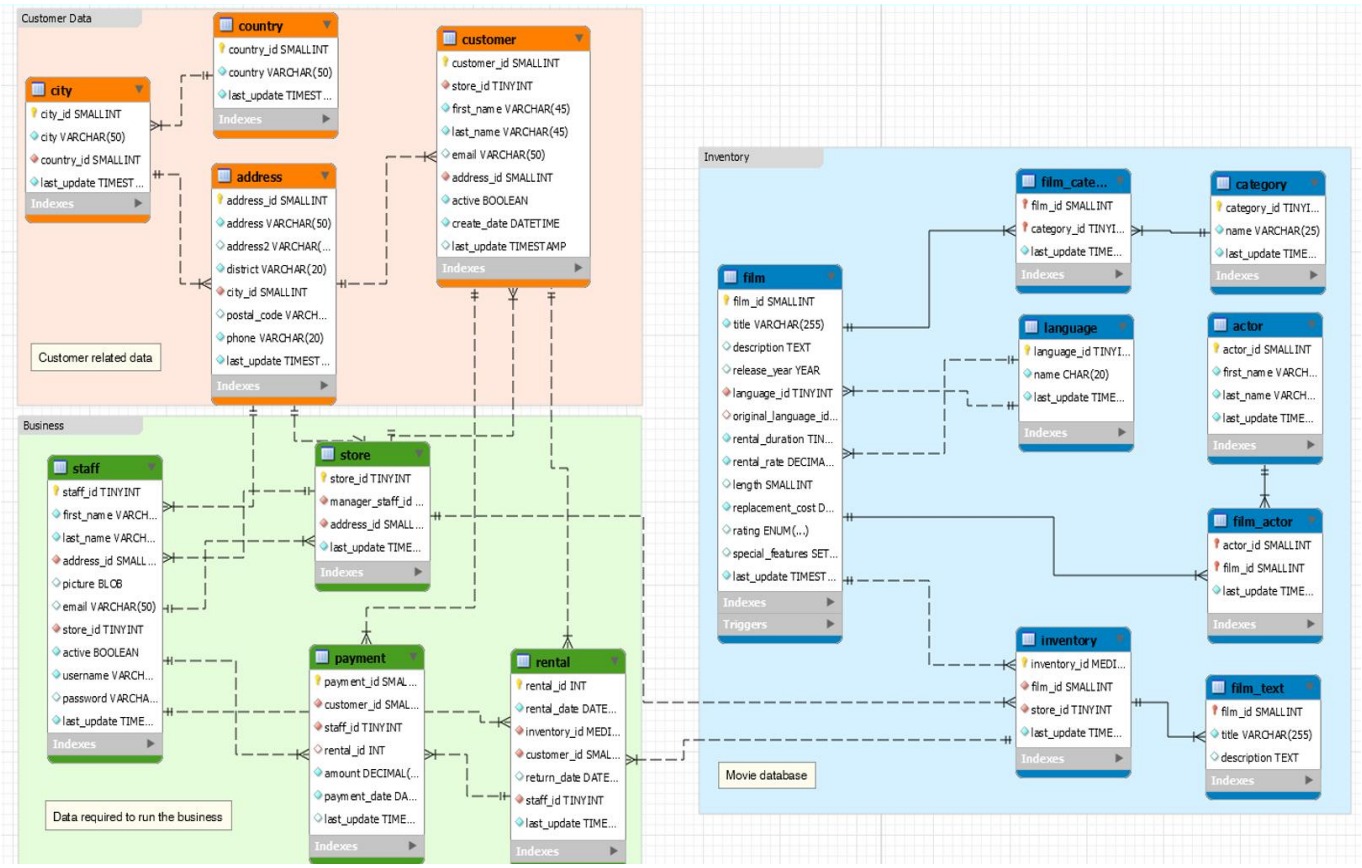

Figure 1: CineFlix Entity-Relationship Diagram

Customer Domain	**Business Domain**	**Inventory Domain**
city, country, address, customer	staff, store, payment, rental	film, actor, category, language, inventory, film_actor, film_category, film_text

3. Schema Reference — All Tables & Columns

Below is the full column-level reference for every table in the CineFlix schema. PK = Primary Key, FK = Foreign Key.

film

Column Name	Data Type	Key	Description
film_id	SMALLINT	PK	Unique film identifier
title	VARCHAR(255)		Film title
description	TEXT		Film plot summary
release_year	YEAR		Year of release
language_id	TINYINT	FK	Ref: language
original_language_id	TINYINT	FK	Ref: language (original)
rental_duration	TINYINT		Default rental days
rental_rate	DECIMAL(4,2)		Daily rental price
length	SMALLINT		Duration in minutes
replacement_cost	DECIMAL(5,2)		Cost to replace film
rating	ENUM		MPAA rating (G/PG/R...)
special_features	SET		Extras (Trailers/Deleted...)
last_update	TIMESTAMP		Last row update time

customer

Column Name	Data Type	Key	Description
customer_id	SMALLINT	PK	Unique customer ID
store_id	TINYINT	FK	Ref: store
first_name	VARCHAR(45)		Customer first name
last_name	VARCHAR(45)		Customer last name
email	VARCHAR(50)		Customer email address
address_id	SMALLINT	FK	Ref: address
active	BOOLEAN		1 = active account
create_date	DATETIME		Account creation date
last_update	TIMESTAMP		Last row update

rental

Column Name	Data Type	Key	Description
rental_id	INT	PK	Unique rental transaction ID

Column Name	Data Type	Key	Description
rental_date	DATETIME		Date/time of rental
inventory_id	MEDIUMINT	FK	Ref: inventory
customer_id	SMALLINT	FK	Ref: customer
return_date	DATETIME		Date/time of return (NULL = open)
staff_id	TINYINT	FK	Ref: staff who handled
last_update	TIMESTAMP		Last row update

payment

Column Name	Data Type	Key	Description
payment_id	SMALLINT	PK	Unique payment ID
customer_id	SMALLINT	FK	Ref: customer
staff_id	TINYINT	FK	Ref: staff who collected
rental_id	INT	FK	Ref: rental
amount	DECIMAL(5,2)		Payment amount in USD
payment_date	DATETIME		Date/time of payment
last_update	TIMESTAMP		Last row update

inventory

Column Name	Data Type	Key	Description
inventory_id	MEDIUMINT	PK	Unique inventory item ID
film_id	SMALLINT	FK	Ref: film
store_id	TINYINT	FK	Ref: store
last_update	TIMESTAMP		Last row update

staff

Column Name	Data Type	Key	Description
staff_id	TINYINT	PK	Unique staff ID
first_name	VARCHAR(45)		Staff first name
last_name	VARCHAR(45)		Staff last name
address_id	SMALLINT	FK	Ref: address
email	VARCHAR(50)		Staff email
store_id	TINYINT	FK	Ref: store
active	BOOLEAN		1 = active employee
username	VARCHAR(16)		Login username
password	VARCHAR(40)		SHA1 hashed password

Column Name	Data Type	Key	Description
last_update	TIMESTAMP		Last row update

store

Column Name	Data Type	Key	Description
store_id	TINYINT	PK	Unique store ID
manager_staff_id	TINYINT	FK	Ref: staff (manager)
address_id	SMALLINT	FK	Ref: address
last_update	TIMESTAMP		Last row update

category

Column Name	Data Type	Key	Description
category_id	TINYINT	PK	Unique genre ID
name	VARCHAR(25)		Genre name (e.g. Action)
last_update	TIMESTAMP		Last row update

actor

Column Name	Data Type	Key	Description
actor_id	SMALLINT	PK	Unique actor ID
first_name	VARCHAR(45)		Actor first name
last_name	VARCHAR(45)		Actor last name
last_update	TIMESTAMP		Last row update

film_actor (Junction)

Column Name	Data Type	Key	Description
actor_id	SMALLINT	**PK / FK**	Ref: actor
film_id	SMALLINT	**PK / FK**	Ref: film
last_update	TIMESTAMP		Last row update

film_category (Junction)

Column Name	Data Type	Key	Description
film_id	SMALLINT	**PK / FK**	Ref: film
category_id	TINYINT	**PK / FK**	Ref: category
last_update	TIMESTAMP		Last row update

language

Column Name	Data Type	Key	Description
language_id	TINYINT	PK	Unique language ID
name	CHAR(20)		Language name
last_update	TIMESTAMP		Last row update

address

Column Name	Data Type	Key	Description
address_id	SMALLINT	PK	Unique address ID
address	VARCHAR(50)		Street address line 1
address2	VARCHAR(50)		Street address line 2
district	VARCHAR(20)		District or state
city_id	SMALLINT	FK	Ref: city
postal_code	VARCHAR(10)		ZIP / postal code
phone	VARCHAR(20)		Contact phone number
last_update	TIMESTAMP		Last row update

city

Column Name	Data Type	Key	Description
city_id	SMALLINT	PK	Unique city ID
city	VARCHAR(50)		City name
country_id	SMALLINT	FK	Ref: country
last_update	TIMESTAMP		Last row update

country

Column Name	Data Type	Key	Description
country_id	SMALLINT	PK	Unique country ID
country	VARCHAR(50)		Country name
last_update	TIMESTAMP		Last row update

film_text (FTS Cache)

Column Name	Data Type	Key	Description
film_id	SMALLINT	PK	Ref: film (sync'd)
title	VARCHAR(255)		Film title (full-text index)
description	TEXT		Plot (full-text index)

4. Business Queries & SQL Solutions

Query 1: Most Popular Film Genre by Total Rentals

Business Problem

The procurement team needs to know which genre drives the most rental activity across all stores, so they can prioritize reordering and marketing budget allocation.

SQL Query

```
# 1
SELECT
    c.name                    AS genre, #C = category
    COUNT(r.rental_id)        AS total_rentals
FROM rental r
JOIN inventory i   ON r.inventory_id  = i.inventory_id
JOIN film_category fc ON i.film_id    = fc.film_id
JOIN category c    ON fc.category_id  = c.category_id
GROUP BY c.name
ORDER BY total_rentals DESC
LIMIT 10;
```

Output

genre	total_rentals
Sports	1179
Animation	1166
Action	1112
Sci-Fi	1101
Family	1096
Drama	1060
Documentary	1050
Foreign	1033
Games	969
Children	945

Query 2: Customers Who Rented More Than the Average

Business Problem

The loyalty programme team wants to identify high-engagement customers (those renting above the portfolio average) for a premium membership upgrade campaign.

SQL Query

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name)  AS customer_name,
    COUNT(r.rental_id)                     AS total_rentals
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(r.rental_id) > (
    SELECT AVG(rental_count)
    FROM (
        SELECT COUNT(rental_id) AS rental_count
        FROM rental
        GROUP BY customer_id
    ) sub
)
ORDER BY total_rentals DESC;
```

Output

customer_id	customer_name	total_rentals
148	ELEANOR HUNT	46
526	KARL SEAL	45
236	MARCIA DEAN	42
144	CLARA SHAW	42
75	TAMMY SANDERS	41
197	SUE PETERS	40
469	WESLEY BULL	40
468	TIM CARY	39
137	RHONDA KENNEDY	39
178	MARION SNYDER	39
459	TOMMY COLLAZO	38
295	DAISY BATES	38
5	ELIZABETH BROWN	38
410	CURTIS IRBY	38
257	MARSHA DOUGLAS	37
176	JUNE CARROLL	37
198	ELSIE KELLEY	37
366	BRANDON HUEY	37
267	MARGIE WADE	36
439	ALEXANDER FENNELL	36
354	JUSTIN NGO	36

Query 3: Total Spend per Customer (Including Zero-Spend)

Business Problem

Finance needs a complete customer billing report that includes customers with no rental history, so that dormant accounts can be identified and targeted with win-back promotions.

SQL Query

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name)  AS customer_name,
    COALESCE(SUM(p.amount), 0.00)           AS total_spent
FROM customer c
LEFT JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spent DESC;
```

Explanation

LEFT JOIN ensures every customer row is preserved even when no matching payment exists. COALESCE converts NULL (no payments) to 0.00 for clean numeric output. SUM aggregates all payments per customer.

Sample Output

customer_id	customer_name	total_spent
526	KARL SEAL	221.55000000000013
148	ELEANOR HUNT	216.5400000000001
144	CLARA SHAW	195.58000000000007
137	RHONDA KENNEDY	194.61000000000007
178	MARION SNYDER	194.61000000000004
459	TOMMY COLLAZO	186.62000000000012
469	WESLEY BULL	177.60000000000002
468	TIM CARY	175.61000000000004
236	MARCIA DEAN	175.58000000000004
181	ANA BRADLEY	174.66000000000005
176	JUNE CARROLL	173.63000000000002
259	LENA JENSEN	170.67000000000002
50	DIANE COLLINS	169.65000000000006
522	ARNOLD HAVENS	167.67000000000002
410	CURTIS IRBY	167.62000000000003
403	MIKE WAY	166.65
295	DAISY BATES	162.62000000000003
209	TONYA CHAPMAN	161.68
373	LOUIS LEONE	161.65
470	GORDON ALLARD	160.68000000000004
187	BRITTANY RILEY	159.72000000000003

Query 4: Film Count per Category (Including Empty Categories)

Business Problem

The content catalogue team needs to know how many films exist in each genre, including any genre with no assigned films, to detect catalogue gaps and plan content acquisition.

SQL Query

```
SELECT
    c.category_id,
    c.name                          AS category_name,
    COUNT(fc.film_id)               AS film_count
FROM category c
LEFT JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.category_id, c.name
ORDER BY film_count DESC;
```

Explanation

LEFT JOIN on the category → film_category junction ensures categories with zero films are included. COUNT on the FK column returns 0 for NULLs, correctly showing empty categories without the need for COALESCE.

Output

category_id	category_name	film_count
15	Sports	74
9	Foreign	73
8	Family	69
6	Documentary	68
2	Animation	66
1	Action	64
13	New	63
7	Drama	62
10	Games	61
14	Sci-Fi	61
3	Children	60
5	Comedy	58
4	Classics	57
16	Travel	57
11	Horror	56
12	Music	51
17	Romance	0
18	Fiction	0
19	War	0
20	Mystery	0

Query 5: Films with Above-Average Rental Rates

Business Problem

The pricing team wants to audit films priced above the portfolio average to validate their premium positioning and identify candidates for promotional discounting.

SQL Query

```
SELECT
    film_id,
    title,
    rental_rate,
    ROUND(rental_rate - (
        SELECT AVG(rental_rate) FROM film
    ), 2)                           AS rate_above_avg
FROM film
WHERE rental_rate > (SELECT AVG(rental_rate) FROM film)
ORDER BY rental_rate DESC, title;
```

Explanation

Scalar subquery in WHERE computes the portfolio average rental_rate in-line. A second scalar subquery in SELECT calculates how far above average each film sits, making the pricing premium immediately visible.

Sample Output

film_id	title	rental_rate	rate_above_avg
1	ACADEMY DINOSAUR	4.99	1.73
8	AIRPORT POLLOCK	4.99	1.73
13	ALI FOREVER	4.99	1.73

Query 6: Revenue Generated per Store

Business Problem

Operations leadership needs a store-by-store revenue breakdown to benchmark performance across locations and allocate operational budgets fairly.

SQL Query

```
SELECT
    s.store_id,
    ci.city                         AS store_city,
    co.country                      AS store_country,
    SUM(p.amount)                   AS total_revenue,
    COUNT(DISTINCT r.rental_id)     AS total_rentals
FROM store s
JOIN address a    ON s.address_id    = a.address_id
JOIN city ci      ON a.city_id       = ci.city_id
JOIN country co   ON ci.country_id   = co.country_id
JOIN staff st     ON s.store_id      = st.store_id
JOIN payment p    ON st.staff_id     = p.staff_id
JOIN rental r     ON p.rental_id     = r.rental_id
```

```
GROUP BY s.store_id, ci.city, co.country
ORDER BY total_revenue DESC;
```

Explanation	Joins the full geographical chain (store → address → city → country) for human-readable store names. Payments are attributed to a store via the staff member who processed them, ensuring revenue is correctly bucketed.

Sample Output

store_id	store_city	total_revenue	total_rentals
2	Woodridge	$33,726.77	7,311
1	Lethbridge	$33,089.74	7,160

Query 7: Top 10 Actors by Film Appearances

Business Problem	The marketing team wants to know which actors appear in the most films so they can use talent recognition in promotional materials and franchise-style marketing campaigns.

SQL Query

```
SELECT
    a.actor_id,
    CONCAT(a.first_name, ' ', a.last_name)  AS actor_name,
    COUNT(fa.film_id)                       AS film_count
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name
ORDER BY film_count DESC
LIMIT 10;
```

Explanation	Simple JOIN through the film_actor junction table with GROUP BY and COUNT. The junction table pattern means each row represents one actor-film pairing, so COUNT(film_id) directly gives appearances.

Sample Output

actor_id	actor_name	film_count
107	Gina Degeneres	42
102	Walter Torn	41
198	Mary Keitel	40
181	Matthew Carrey	39

Query 8: Currently Unreturned Rentals (Open Returns)

Business Problem

The operations team needs a live list of all rentals where the film has not yet been returned, to follow up with customers and recover inventory before incurring replacement costs.

SQL Query

```
SELECT
    r.rental_id,
    r.rental_date,
    f.title                          AS film_title,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    c.email,
    DATEDIFF(CURDATE(), r.rental_date)      AS days_out
FROM rental r
JOIN inventory i  ON r.inventory_id  = i.inventory_id
JOIN film f       ON i.film_id       = f.film_id
JOIN customer c   ON r.customer_id   = c.customer_id
WHERE r.return_date IS NULL
ORDER BY days_out DESC;
```

Explanation

Filters on return_date IS NULL to find open rentals only. DATEDIFF(CURDATE(), rental_date) calculates how many days the item has been out, with the most overdue items at the top for immediate follow-up.

Sample Output

rental_id	film_title	customer_name	days_out
11496	HAWKS ALTER	Sally Pierce	6821
11541	MASSACRE USUAL	Dee Poe	6821
11563	WIFE TURN	Yvonne Watkins	6820

Query 9: Inventory Utilisation Rate per Film per Store

Business Problem

Store managers need to know which films are rented most relative to the number of copies on hand, so they can rebalance stock between stores and retire underperforming titles.

SQL Query

```
SELECT
    f.title,
    i.store_id,
    COUNT(DISTINCT i.inventory_id)          AS copies_available,
    COUNT(r.rental_id)                      AS times_rented,
    ROUND(
```

```
        COUNT(r.rental_id) /
        NULLIF(COUNT(DISTINCT i.inventory_id), 0),
        1
    )                                       AS rentals_per_copy
FROM film f
JOIN inventory i   ON f.film_id        = i.film_id
LEFT JOIN rental r ON i.inventory_id   = r.inventory_id
GROUP BY f.film_id, f.title, i.store_id
ORDER BY rentals_per_copy DESC
LIMIT 15;
```

Explanation NULLIF prevents division-by-zero if a store has no copies. The LEFT JOIN on rental preserves films with no rentals. The ratio rentals_per_copy is the utilisation metric: a high value signals undersupply, a low value signals overstock.

Sample Output

title	store_id	copies_available	times_rented	rentals_per_copy
BUCKET BROTHERHOOD	1	2	34	17.0
ROCKETEER MOTHER	2	2	30	15.0
FORWARD TEMPLE	1	1	27	27.0

Query 10: Monthly Revenue Trend

Business Problem The finance director needs a rolling monthly revenue report to track seasonal demand patterns, benchmark year-over-year growth, and forecast cash flow for the next quarter.

SQL Query

```
SELECT
    YEAR(payment_date)              AS payment_year,
    MONTH(payment_date)             AS payment_month,
    DATE_FORMAT(payment_date, '%b %Y') AS month_label,
    COUNT(payment_id)               AS transactions,
    ROUND(SUM(amount), 2)           AS monthly_revenue,
    ROUND(AVG(amount), 2)           AS avg_transaction_value
FROM payment
GROUP BY
    YEAR(payment_date),
    MONTH(payment_date),
    DATE_FORMAT(payment_date, '%b %Y')
ORDER BY payment_year, payment_month;
```

Explanation YEAR() and MONTH() extract the time components for grouping. DATE_FORMAT provides a readable label. Three aggregates (COUNT, SUM,

AVG) give the transaction volume, total revenue, and basket size per month in a single pass.

Sample Output

month_label	transactions	monthly_revenue	avg_transaction_value
May 2005	4824	$14,949.09	$3.10
Jun 2005	9340	$28,373.89	$3.04
Jul 2005	6709	$20,397.78	$3.04

5. Key Relationship Map

The table below summarises every important foreign-key join path used in the queries above. Use this as a quick-reference when writing new queries.

From Table	To Table	Join Condition
rental	inventory	`rental.inventory_id = inventory.inventory_id`
rental	customer	`rental.customer_id = customer.customer_id`
rental	staff	`rental.staff_id = staff.staff_id`
inventory	film	`inventory.film_id = film.film_id`
inventory	store	`inventory.store_id = store.store_id`
film_category	film	`film_category.film_id = film.film_id`
film_category	category	`film_category.category_id = category.category_id`
film_actor	film	`film_actor.film_id = film.film_id`
film_actor	actor	`film_actor.actor_id = actor.actor_id`
payment	rental	`payment.rental_id = rental.rental_id`
payment	customer	`payment.customer_id = customer.customer_id`
store	address	`store.address_id = address.address_id`
address	city	`address.city_id = city.city_id`
city	country	`city.country_id = country.country_id`
customer	address	`customer.address_id = address.address_id`
staff	store	`staff.store_id = store.store_id`

6. Appendix — SQL Best Practices Applied

The following SQL engineering conventions are applied consistently throughout this project:

JOIN vs Subquery: JOINs are preferred for set-based filtering; correlated subqueries are used only in HAVING and WHERE clauses where a scalar value comparison is required.

LEFT JOIN for completeness: Wherever the business question requires 'including rows with no matches' (e.g. customers with no rentals, categories with no films), LEFT JOIN is used instead of INNER JOIN.

COALESCE for NULL safety: All SUM/AVG expressions that could produce NULL from a LEFT JOIN are wrapped in COALESCE to return 0 or 0.00 as appropriate.

NULLIF for division safety: Division operations use NULLIF(denominator, 0) to prevent divide-by-zero errors on edge-case data.

Aliases for readability: All derived columns and tables use meaningful aliases so results are immediately legible in BI tools and reports.

Deterministic ORDER BY: All paginated or ranked queries include a deterministic secondary sort (e.g. title) to guarantee stable result ordering.